

Agentic Engineering

Principles

Power is nothing without control



Pirelli, 1995 US TV commercial

"Using AI agents makes it possible for teams who are not deeply technical in the syntax of a language to ship production code. But that team had better be very deeply technical in managing the structure and quality of the code that is produced.

*What the agents give us is the ability to disengage from deep syntax. But **they do not give us the ability to disengage from modular design and architecture.** You still need to be deeply technical in those topics in order to produce good production quality code."*

— Uncle Bob Martin, creator of three out of five SOLID programming principles, 2026

Many more, much smaller steps (MMMSS)

Geepaw Hill

- Break tasks into tiny, independently shippable increments rather than taking large leaps toward a goal, ultimately helping to build robust, iterative software without breaking things.

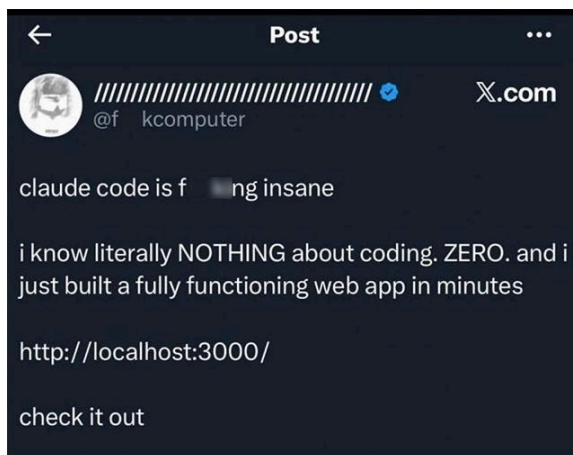
Concepts

1. Human-scope: Architecture and Knowledge Domain

- LLMs lack any knowledge of what is not public on the internet, as in ongoing/private projects or codebases.
- LLMs lack human common sense due to their lack of real-world physical experience.
- LLMs are not deterministic by their own definition. They focus on response averaging and supervised-learning weighted training. This training does not always come from the most reliable sources — i.e., poorly compensated supposed experts from all around the world. That said, continuous usage of those online-based tools and humans flagging wrong results are dynamically "re-training" and sensibly improving their output to high-reliability levels.
- Humans are accountable and will double down on efforts to assure the quality of deployment if their names and jobs are on the line. LLMs will just apologize.
- Humans are required to start processes and have the discernment to assess, shut down, or revert any missteps when agents simply fail to realize — or are incapable of accessing, due to security restrictions — some obvious issue.

2. AE (Agentic Engineering) vs. Vibe Coding

- **Vibe Coding:** A prompt-driven approach to software development where the description of a software algorithm or project is described in natural human language, while an LLM (large language model) AI handles the code generation, debugging, and even, when requested, deployment. It can be done by anyone regardless of technical background.
- **Agentic Engineering:** A professional discipline where system architects and developers design and apply a structured, AI-powered system with strict testing, validation loops, and clear constraints to safely ship production-ready software. It focuses on the project architecture and the workflow design around the AI Coding Agent (LLM Assistant). The human orchestrator retains full responsibility for architecture, code quality, and engineering judgment.



Don't be *that* guy!

3. Agentic Engineering Techniques

3.1. Harness Engineering

Definition: Everything outside the model: instructions, tools, environment, state management, and verification feedback.,

- Constrain agent behavior with explicit rules and boundaries.
- Maintain context across long-running, multi-session tasks.
- Stop agents from declaring victory too early.
 - The agent says "I'm done" when it is not done — this is the most common failure mode.
- Verify work using full-pipeline tests, self-reflection and *adversarial reviews* (competing LLM).
- Make runtime observable and debuggable.

3.2. Spec-driven Development

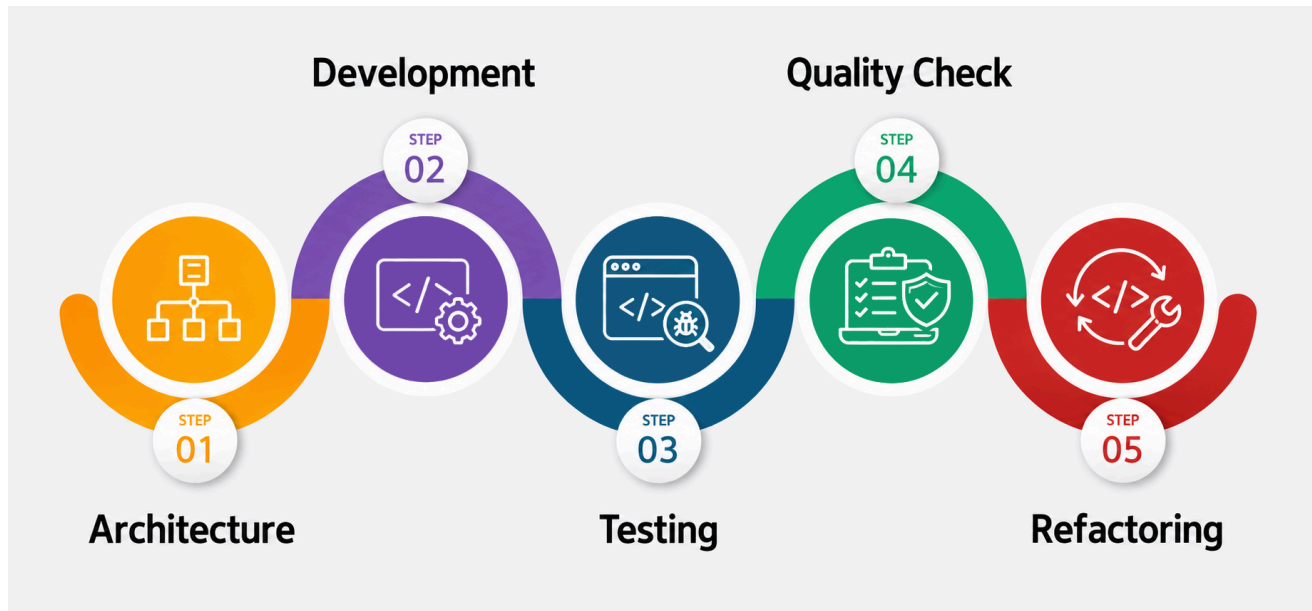
An AE approach largely used up to these days (Sep 2026). It is more focused on feature completion than on architecture compliance and stability. It has already been made somewhat redundant due to AI swarms and other control techniques, which is what Harness Engineering is about.

3.3. Test-driven Development vs. Behavior-driven Development

- **Traditional TDD**, through the "red → green → refactor" cycle, recommends that features should only be implemented after their tests are written for any unit. Considering a "unit" a class, this drastically slows down iteration of execution/gameplay tests, or even AI delivery output, which increases cost correspondingly due to the too-granular scope, making it impractical for most cases. Still, TDD is recommended for critical functionality, such as network-related functionality, or very low-level functionality, such as API or compiler development.
- **BDD (Behavior-driven Development)** focuses on alignment with user behavior and business requirements, making it a perfect fit for LLM-assisted engineering. It suggests that for any surfacing bug, a test should be written first before trying to fix the bug. Major, feature-specific tests may also be written and checked by the LLM during implementation and continuously rechecked after initial release.

Feature Development Lifecycle

Inspired by the XP (Extreme Programming) development cycle



1. Architecture Phase

Provide the Project Architecture and Skeleton of the Project (Human)

- Set up the agentic behavior and LLM "system variables":
 - AGENTS.md (Codex) or CLAUDE.md (Claude Code)
 - Highest-ROI step
- Project overview and purpose: one sentence.
- Complete tech stack:
 - Unreal-specific: Engine version
 - Build-platform compatibility
 - Render Pipeline: URP, HDRP
 - Frontend-backend integration
- General architectural vision:
 - Unreal-specific: Singleton, Data Assets, Web-provided data
 - Separation of GUI and game logic. E.g., MVC.
- Content directory structure.
- Services and models:
 - Unreal-specific: plugins in use
- Design patterns
- Post-implementation checklist.

Reference: Akita on Rails

Flesh out the Project or Feature (AI)

Fill-in the gaps of the project or feature with the required classes, method calls, and interfaces.

2. Development Phase: Behavior-driven Development (BDD)

Define Granularity, Coverage, and Test Descriptions (Human)

- Define and assure test execution and acceptance for each type of task.
- Implement skills for each type of task, e.g., new panel, new game module, etc.
- Skills should have pre-existing human code references as a guide.

Reinforce the Harness

Aka. “do not let it build junior-level code”.

Hints:

- Request the agent to report its plan before implementation. (use 'Planning' mode first if Agent has it)
- Write tests for all new bug fixes before implementing those fixes.
- Never commit or push without permission.
- When changing a method, check the cyclomatic complexity and report it if it is greater than 5.
- All uses of tools, e.g., `sed` and `grep`, are allowed.
- Methods with fewer than 10 lines should generally be avoided in favor of commented sections, unless that logic block is actively reused in the same source file.
- Creation of and addition to external tools or utility libraries should require approval by the human orchestrator.
- Break into separate functions any methods with broad semantic coverage - breaking the single-responsibility principle (SRP).
- Add explicit syntax and naming conventions, e.g.:
 - Functions with more than 4 words should be avoided.
 - Variables, including fields and properties, and parameters should stay within the 4-word limit and use camelCase;
 - Use Epic's C++ coding standard
 - Define method/scope braces and spacing formatting.
 - `return` commands should always be on their own separate line.
 - Use Unreal-specific data structures and methods over their generic C++ counterparts.
 - Example: `FMath::IsNearlyEqual(a, b)` vs. `std::abs(a - b) < 0.0001f`

Set Up the Git/Commit Environment

- **Git worktree:** Allows you to have multiple branches of the same repository checked out simultaneously in separate directories. Worktrees let you work on different features or bug fixes in parallel without stashing changes or making multiple full clones of the project.
- **Automatic branches:** Assure the AI won't implement any new tasks without creating a new branch. Reinforce that it will not touch branches it did not create. Overview code merges.

Agent Tasks (AI)

1. Create the BDD tests, focused on actual user flow.
2. Churn through code according to the harness.
3. Run and validate tests; go to step 2 if any failure is found.
4. Update the `BUGFIX.md` file with corrected bugs and the approach used to fix them.

3. Testing Phase

Principle: Success should be measured by real outcomes — lower defect rates, faster development cycles, and clearer feature delivery — not just test coverage numbers.

Define Environment Security (Human)

- **AI-Jail:** multiple layers of isolation for optimal safety. Too complex for average use.
- **Antigravity Workspace / Codex Project Isolation Sandbox:** Project-folder and ~/.antigravity/ root-directory restriction.
 - Multi-root Workspace: File > Add Folder to Workspace
- **Docker/podman container**
- **System-level kernel guard (Codex)**
 - Claude employs a weaker "permission model."
 - Since October 2025, Claude Code has offered a runtime sandbox via the /sandbox command. Underneath, it uses Bubblewrap on Linux and sandbox-exec on macOS.

Define explicit verification commands:

```
pytest tests/api/v2/ && python -m mypy src/
```

Agent Tasks (AI)

- Automatically run linters and ensure the output is clean.
 - A linter is a language-specific developer tool that analyzes source code against predefined rules to identify errors, vulnerabilities, and stylistic issues before code reaches production.

4. Code Quality Review

General

Assess objective metrics versus reading code as often as possible. Map the functionality through dynamic graphs and reserve code-reading for critical tasks.

Cyclomatic Complexity

Identify complex code segments. It does not measure cognitive complexity.

Cyclomatic complexity is a software metric that quantifies the number of linearly independent paths through a program's source code. Developed by Thomas McCabe in 1976, it is widely used to evaluate code maintainability and estimate the minimum number of unit tests required to achieve full branch coverage.

Code structures that increase complexity:

- if statements and else branches
- Ternary operators: ? :
- switch statements and case blocks
- Loops: for, while, do-while
- Logical operators: &&, ||

Cyclomatic complexity dictates how many unique unit tests are needed to cover every possible execution path.

- **1–10:** Simple, highly testable, and maintainable code. <4 is ideal.
- **11–15:** Moderately complex; requires a moderate amount of testing.
- **>15:** Highly complex; strongly recommended to refactor or split into smaller functions; often indicates architectural bottlenecks.

Test Coverage

The percentage of code that is being evaluated by one of the predefined tests.

Dependency Structure / Dependency Metrics (Graph/Matrix)

The goal should always be high cohesion and low coupling.

Deep-nested dependencies and cyclic dependencies

Hints at centralization and systemic dependency. Complex code is harder to maintain by both humans and LLMs.

Mutation Testing

Automatically changes conditions, such as switching < to >, to generate faults, or mutations, in the code, then runs the tests. If tests fail, then the mutation is killed; if tests pass, then the mutation lived. The higher the kill ratio, the higher the code quality. All surviving mutants must be killed, and all uncovered mutants must be covered with tests.

Mutation testing takes it a step further than test coverage by ensuring that the test coverage is real coverage with real assertions behind it.

C.R.A.P. (Change Risk Anti-Patterns)

A formula that takes into account cyclomatic complexity and test coverage. The result must be lower than 8.

```
GitHub action | npx skills@latest add mohganji/skills
```

5. Refactoring

Code Refactoring (AI)

Aggressively search for repetitive code that can be generalized into a single function or moved into a shared tool library.

Optimization Refactor (Human)

Carry out performance/profiling tests and ask the AI to produce A-B test compilations. Feed back the results to the AI for further optimization.

- Do not try to review AI code as if it is human code.
- Ask the AI to build graphs explaining the main logic flows.
- Validate the key logic flows and order changes when needed.

Agent-specific Information and Suggestions

Sycophancy

- Agents are programmed to try to answer you anyhow first, and to be accurate as a secondary priority. Most LLM Agents will also try to flatter you to keep your attention. That's by design, so disable it as soon as possible!

Context Window Size

- The smart zone → Below approximately 60k tokens
 - The dumb zone → Above 60k tokens
 - How to keep in the smart zone?
 - Split work in multiple agents
 - Compress / Compact [MD files]
 - Create new chats → Fresh context
 - Branch from current chat → Auto-compress - Fresh context
 - HANDOFF skill (matt pockock): more robust and focused than default "compress"
-
- **1M context window:** Better for the initial research phase on large codebases; more prone to hallucination. Suggestion: require final output to an .md file.
 - **200k context window:** Better for feature implementation; requires well-crafted harnessing.